23CSE455- Design Pattern Case Study Instructions

Use your assigned case study. Submit one completed report and one team folder on or before 09/10/2026.

1. patterns.csv

One row = one confirmed pattern collaboration in one source version. Include all applicable participants, not only a main class.

Column	What to enter
source_version	Original code
pattern_name	Exact pattern name, e.g. Observer or Strategy.
pattern_type	Category: Creational, Structural, Behavioral, GRASP, etc.
instance_name	Meaningful name of one collaboration, e.g. Book Return Notification.
participant_roles	Every applicable role mapped to its actual class/interface/function.
source_locations	Each participant: file path and line range.
intent_and_evidence	Describe the problem solved and code interactions proving the pattern.

Observer example: Subject=not separately declared; ConcreteSubject=Library (also performs subject role); Observer=ReturnListener; ConcreteObserver=NotificationLog. If roles are combined or absent in the code, state that; do not invent classes.

2. pattern_counts.csv - counts from your pattern records

Column	What to enter
source_version	Code version being counted.
summary	per_pattern, total_pattern_types or total_pattern_instances.
pattern_name	Pattern name for per_pattern rows; blank for total rows.
count	Number of instances of that pattern, distinct pattern types or total instances.

Count each source version separately, excluding removed instances from its current totals. Two concrete fine-policy classes in one collaboration mean one Strategy instance. Do not add before/after observations as new instances.

3. patterns_evolution.csv

One row = one confirmed pattern collaboration in one source version. Include all applicable participants, not only a main class.

Column	What to enter
source_version	Original code or code after Change1/Change2/Change3; not a submission stage.
pattern_name	Exact pattern name, e.g. Observer or Strategy.
pattern_type	Category: Creational, Structural, Behavioral, GRASP, etc.
instance_name	Meaningful name of one collaboration, e.g. Book Return Notification.
participant_roles	Every applicable role mapped to its actual class/interface/function.
source_locations	Each participant: file path and line range.
intent_and_evidence	Describe the problem solved and code interactions proving the pattern.
status_after_change	baseline, preserved, modified, extended, replaced, removed or newly_introduced.
change_description	Briefly state added/changed/removed participants and the reason.

After each change, append some column to the same pattern.csv and named as pattern_evolution.csv. Keep original rows and the same instance name for the continuing collaboration. Update participants, locations, evidence and status. Retain removed-instance rows with status removed.

4. cross_language.csv - snippet comparison

One row = one representative confirmed pattern instance implemented in one language. For each detected pattern type, implement its small problem in all four languages, using the same inputs and expected outputs.

Column	What to enter
pattern_name	Confirmed pattern implemented.
instance_name	Representative source collaboration used for the snippet.
language	Java, Python, JavaScript or C++.
snippet_path	Production code file; keep tests separate.
source_loc	Production lines excluding blank lines, comments and tests.
max_function_cc	Maximum function cyclomatic complexity; record the counting rule/tool in README.
runtime_compiler	Runtime/compiler name and version.
test_command	Command to compile/run/test; working folder in README.
test_result	Actual passed/total or failed result; not_run with reason if unavailable.
median_process_ms	Measured median elapsed time in milliseconds; blank if not run.
language_notes	Language-specific structure and observed static/dynamic differences.

Timing: compile first; one warmup, then five measured runs on the same machine; report the median. State once in README what time includes (startup/test input reading or only the snippet). Keep raw results as evidence. Never estimate missing execution time.

5. llm.csv - AI use and verification

Column	What to enter
part	Pattern discovery, evolution or cross-language analysis.
model_and_date	Displayed model and actual interaction date.
prompt_file	Path to the exact prompt.
input_file	Path to the supplied code/context.
output_file	Path to the raw model output.
evaluation	Correct/partial/incorrect/cannot confirm, with concise correction.
verification_file	Path to source/test evidence supporting your evaluation.

Required file upload

Use the supplied column names, one header row and UTF-8 CSV. Quote cells containing commas. Use paths relative to your team folder. Verify roles and counts against source. Record application, versions, inspected modules and measurement rules once in README;.

Upload to github.com/aswathymohan-amrita/23CSE455_DP_CaseStudy in your own A1/A2/B1/etc. folder: one report PDF + LaTeX source; README; List students by roll number, name and batch (CSE A, EAC , AIDS,AIE etc.). There are no separate part-wise submissions.

Complete repository hierarchy

Repository: `https://github.com/aswathymohan-amrita/23CSE455_DP_CaseStudy`

Create your team folder directly at the repository root. The expanded A1 folder below is the example for every team. Replace A1 with your exact team name. Submit this entire folder once next week.

```
23CSE455_DP_CaseStudy/
|-- A1/
|   |-- README.md
|   |-- A1_Report.pdf
|   |-- A1_Report.tex
|   |-- data/
        |-- code                      original application source
        |--test_result                testcases, screenshort..etc
    |-- Part1/
|   |   |-- patterns.csv
|   |   |-- Pattern_count.csv
|   |   |-- pattern_architecture         diagram source + image
|   |   |--Part1_prompt.txt
|   |   |‘-- llm.csv
    |-- Part2/
    |    |-- code/                    evolved application source
|   |   |-- changes/
|   |   |   |   |-- Change1/         requirement, diff, test result
|   |   |   |   |-- Change2/         requirement, diff, test result
|   |   |   |   ‘-- Change3/         requirement, diff, test result
|   |   |-- pattern_evalution.csv
|   |   |---- Pattern_count.csv
|   |   |--diagrams                  updated UML where needed
    |   |-- Part2_prompt.text
|   |   ‘-- llm.csv

|   |-- Part3/
|   |   |--pattern_problem_name/     repeat for each pattern type
|       |       |-- java/            snippet, test runner, README
|       |       |-- python/          snippet, test runner, README
|       |       |-- javascript/      snippet, test runner, README
|       |       ‘-- cpp/             snippet, test runner, README
|   |   ‘-- llm/
|   |           |-- part3_prompt.txt
|   |           |-- llm.csv
|   |           |--pattern_crosslanguage.csv
|   |           ‘--A1_slide.pdf
|-- A2/                             same structure for team A2
|-- ...                             remaining A teams
|-- B1/                             same structure for team B1
|-- B2/                             same structure for team B2
‘-- ...                             remaining B teams
```

Keep one report and one dataset folder. Part1, Part2 and Part3 organize the supporting code and evidence; they are not separate submission stages. Do not repeat the four CSVs inside them.

README.md: include application/source URL, source commits, environment versions, run/test commands, inspected modules, LOC/complexity/timing rules and limitations. Each language README states how to run that snippet and its tests.

File references: CSV paths start from your team folder, for example `Part1/code/Library.java` or `Part3/evidence/test_re`
Keep original and changed source versions reproducible through the recorded commits or saved versions. Do not upload passwords, dependency caches or compiled binaries.